



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DA BAHIA

PEDRO VICTOR HIPÓLITO CABRAL

**CENTRO DE AJUDA: SISTEMA INTELIGENTE DE REDIRECIONA-
MENTO DE COMUNICAÇÃO EM SITUAÇÕES EMERGENCIAIS**

SANTO ANTÔNIO DE JESUS - BA

2026

PEDRO VICTOR HIPÓLITO CABRAL

**CENTRO DE AJUDA: SISTEMA INTELIGENTE DE REDIRECIONA-
MENTO DE COMUNICAÇÃO EM SITUAÇÕES EMERGENCIAIS**

Trabalho de Conclusão de Curso apresentado a Instituto Federal de Educação, Ciência e Tecnologia da Bahia, campus Santo Antônio de Jesus - BA, como requisito parcial para obtenção do grau de Tecnólogo em Análise e Desenvolvimento de Sistemas.

Prof. Leandro Costa Souza

SANTO ANTÔNIO DE JESUS - BA

2026

PEDRO VICTOR HIPÓLITO CABRAL

**CENTRO DE AJUDA: SISTEMA INTELIGENTE DE REDIRECIONA-
MENTO DE COMUNICAÇÃO EM SITUAÇÕES EMERGENCIAIS**

A banca examinadora, abaixo listada, aprova o Trabalho de Conclusão de Curso **CENTRO DE AJUDA: SISTEMA INTELIGENTE DE REDIRECIONAMENTO DE COMUNICAÇÃO EM SITUAÇÕES EMERGENCIAIS** elaborado por **PEDRO VICTOR HIPÓLITO CABRAL** como requisito parcial para obtenção do grau de Tecnólogo em Análise e Desenvolvimento de Sistemas, pelo Instituto Federal de Educação, Ciência e Tecnologia da Bahia.

Santo Antônio de Jesus - BA, sexta-feira, 18 de setembro de 2026

Comissão Examinadora

Prof. Mst. Leandro Costa Souza (IFBA)
Orientador

Prof. Dr. (IFBA)

Prof. Dr. (IFBA)

RESUMO

Este trabalho apresenta o desenvolvimento de uma prova de conceito de um sistema inteligente para redirecionamento de solicitações emergenciais, denominado Centro de Ajuda, que utiliza inteligência artificial multimodal para interpretar relatos em texto, áudio e imagem e recomendar o serviço de emergência mais adequado.

Palavras-chave: Inteligência Artificial. Emergência. Multimodal. LLM.

ABSTRACT

This work presents a proof of concept of an intelligent emergency redirection system called Centro de Ajuda, using multimodal AI to interpret text, audio and image reports and recommend the most appropriate emergency service.

Keywords: Artificial Intelligence. Emergency. Multimodal. LLM.

LISTA DE FIGURAS

Figura 1 – Fluxo de interação entre usuário, servidor e inteligência artificial	16
Figura 2 – Fluxo da API Fetch com Promise	32
Figura 3 – Fluxo de comunicação entre frontend, backend e APIs externas	33
Figura 4 – Arquitetura da aplicação cliente	34
Figura 5 – Tela inicial do Centro de Ajuda	35
Figura 6 – Tela de espera	40
Figura 7 – Tela de resposta com recomendação	40
Figura 8 – Arquitetura geral do sistema	48

LISTA DE TABELAS

Tabela 1 – Comparação de modelos quanto aos critérios técnicos	29
Tabela 2 – Cenários testados	49
Tabela 3 – Exemplo de métricas por modelo	49
Tabela 4 – Comparação por critérios — modelo isolado	50
Tabela 5 – Viabilidade por ecossistema (composição sequencial)	50

LISTA DE CÓDIGOS

Código 1 –	Campo de texto da ocorrência	35
Código 2 –	Captura de texto via DOM	36
Código 3 –	Solicitação de acesso ao microfone	36
Código 4 –	Gravação de áudio com MediaRecorder	37
Código 5 –	Captura de imagem	38
Código 6 –	Envio da ocorrência ao servidor	39
Código 7 –	Tratamento da resposta do servidor	41
Código 8 –	Redirecionamento para ligação	42
Código 9 –	Inicialização do servidor	42
Código 10 –	Configuração Express com CORS e bodyParser	43
Código 11 –	Geocodificação reversa	44
Código 12 –	System Prompt	45
Código 13 –	Assistant Prompt contextual	45
Código 14 –	Montagem conteúdo multimodal	45
Código 15 –	Chamada OpenRouter	46
Código 16 –	Tratamento resultado e validação JSON	47

LISTA DE ABREVIATURAS E SIGLAS

CNNS	Convolutional Neural Networks
LLMS	Large Language Models
RNNS	Recurrent Neural Networks
SAMU	Serviço de Atendimento Móvel de Urgência

SUMÁRIO

1 Introdução	13
1.1 <i>Justificativa</i>	13
1.1.1 O que é o número de emergência universal	13
1.1.2 Números de emergência no Brasil	14
1.2 <i>Objetivo</i>	15
1.2.1 Objetivos Específicos	16
1.3 <i>Correlatos</i>	17
1.3.1 RapidSOS	17
1.3.2 911 e 112	18
2 Revisão Bibliográfica	19
2.1 <i>Inteligência Artificial</i>	19
2.1.1 Aprendizado de Máquina	19
2.1.2 Redes Neurais e Deep Learning	20
2.1.3 Arquitetura Transformer	22
2.2 <i>LLMs</i>	23
2.2.1 Engenharia de Prompt	24
2.3 <i>Modelos Multimodais</i>	25
2.3.1 Processamento de Imagem	26
2.3.2 Processamento de Áudio	27
2.3.3 Processamento de Vídeo	28
3 Metodologia	29
3.1 <i>CrITÉrios de Escolha para o(s) Modelo(s) LLM</i>	29
3.1.1 GPT-5.5	29
3.1.2 Claude Opus 4.8	30
3.1.3 Llama 4 Maverick	30
3.1.4 Qwen 3.6 Plus	30

3.1.5 Gemini 3.5 Flash	30
3.1.6 Nemotron 3 Nano Omni	31
3.1.7 North Mini Code	31
3.2 <i>Requisitos do Frontend</i>	31
3.2.1 Interface do Usuário	31
3.2.2 Compatibilidade Multiplataforma	31
3.2.3 Comunicação Cliente-Servidor	32
3.3 <i>Requisitos do Backend</i>	32
3.3.1 Infraestrutura Backend	32
3.3.2 Ambiente de Execução	32
3.3.3 Estrutura da Aplicação	33
3.3.4 Integração com Serviços Externos	33
3.4 <i>Fluxo de Comunicação</i>	33
4 Desenvolvimento	34
4.1 <i>Abordagem Geral</i>	34
4.2 <i>Implementação da Aplicação Cliente</i>	34
4.2.1 Arquitetura do Cliente	34
4.2.2 Tecnologias da Aplicação Cliente	35
4.2.3 Tela Inicial	35
4.2.4 Tela de Espera	40
4.2.5 Tela de Resposta	40
4.3 <i>Implementação do Servidor</i>	42
4.4 <i>Construção do Prompt</i>	44
4.4.1 Formato de Saída (JSON)	46
4.5 <i>Integração com Modelos de Linguagem</i>	46
5 Análise dos Resultados	49
5.1 <i>Modelos e Cenários</i>	49

5.2 Coleta e Comparações	49
6 Conclusão	51
6.1 Avaliação dos Modelos	51
6.2 Propostas Futuras	51
6.3 Considerações Finais	51
REFERÊNCIAS	52
Apêndice A – Códigos Complementares	55
Anexo A – Portaria de Autorização	56

1 Introdução

O acionamento de serviços de emergência no Brasil ainda é marcado pela descentralização dos recursos públicos e privados, causando confusão pública, falhas de acesso, dentre outros fatores que comprometem a qualidade das respostas dos órgãos competentes. Este capítulo define o que é o número de emergência universal, qual tipo de problema é causado quando o serviço não é disponibilizado para a população, e como esse problema pode ser resolvido através de um aplicativo, especialmente quando a máquina pública não disponibiliza tais ferramentas.

1.1 *Justificativa*

Para definir uma estratégia, primeiro é necessário entender os principais desafios que as motivam, nesse cenário, com ênfase às limitações enfrentadas pela população diante do baixo nível de preparo e acesso à informação.

1.1.1 O que é o número de emergência universal

Segundo Moss (2017), o número de emergência universal é, essencialmente, um identificador telefônico único criado para centralizar o acesso a qualquer tipo de urgência. Sua função é permitir que uma pessoa consiga, em qualquer situação crítica, acionar socorro imediato sem precisar distinguir qual órgão é responsável pelo atendimento. Em vez de vários números de emergência competindo por espaço, existe um ponto único centralizado capaz de encaminhar a chamada, através de informações atualizadas e localizadas, criando um canal simples, memorável e intuitivo. A autora descreve o número de emergência universal como um mecanismo que unifica a porta de entrada do sistema, reduzindo barreiras cognitivas, consequentemente facilitando o acionamento rápido e preciso do serviço.

De acordo com Moss (2017), diversos países adotaram modelos de números de emergência universal ao longo das décadas, estruturando sistemas nacionais centrali-

zados de emergência que operam a partir de um único canal de acesso. O Reino Unido, por exemplo, implementou o número 999 como sistema nacional de emergência, enquanto os Estados Unidos consolidaram o uso do famoso 911 para o acionamento de polícia, bombeiros e serviços médicos. No entanto, a implementação dessa ferramenta não ocorreu de forma uniforme em todos os países, existindo diferenças estruturais, tecnológicas e administrativas que impactaram a possibilidade de integração dos serviços unificados de emergência (MOSS, 2017).

1.1.2 Números de emergência no Brasil

No ambiente brasileiro, o atendimento emergencial permanece distribuído entre diversos diferentes números especializados, cada um associado a um órgão que socorre tipos específicos de ocorrência. Entre os principais exemplos estão o 190, destinado ao acionamento da Polícia Militar em situações de violência, crimes e ameaças à segurança pública; o 192, responsável pelo Serviço de Atendimento Móvel de Urgência (SAMU), voltando para emergências médicas; e o 193, destinado ao Corpo de Bombeiros para casos de incêndios, acidentes e resgates. Esse modelo fragmentado busca especializar o atendimento conforme a natureza da ocorrência, concluindo que o cidadão possui plena consciência da existência de múltiplos números de emergência (TELECOMUNICAÇÕES, 2025).

De acordo com a pesquisa de Unijuí (2024), a fragmentação dos números de emergência brasileiros ainda gera dificuldades significativas no reconhecimento e diferenciação dos serviços pela população brasileira. O estudo publicado identificou que, embora o número 190 apresente alto índice de reconhecimento correto (92.2%), há considerável confusão em relação às atribuições do SAMU (192) e do Corpo de Bombeiros (193), que apresentaram índices de reconhecimento correto de apenas 72.6% e 70.8%, respectivamente. Além disso, o estudo aponta que 23% dos entrevistados associaram incorretamente o número 192 ao Corpo de Bombeiros, enquanto 24.2% relacionaram

equivocadamente o número 193 ao SAMU. A pesquisa conclui que parte da população não consegue diferenciar adequadamente os serviços disponíveis, especialmente em situações de estresse, o que pode retardar o acionamento do socorro adequado, consequentemente agravando a ocorrência. Os autores apontam que a unificação dos núcleos de atendimento é uma forte possibilidade para reduzir dúvidas e simplificar o acesso aos serviços emergenciais.

1.2 Objetivo

Devido à existência de múltiplos números de emergência no Brasil e às dificuldades da população em diferenciar corretamente os serviços responsáveis por cada tipo de ocorrência, o presente trabalho tem como objetivo desenvolver uma prova de conceito de um sistema inteligente de redirecionamento de comunicação em situações emergenciais, denominado *Centro de Ajuda*. A proposta consiste em uma aplicação móvel, integrada a um servidor responsável pelo processamento das solicitações realizadas pelos usuários.

A solução permite que o usuário envie relatos em formato textual, sonoro ou visual descrevendo a situação enfrentada, possibilitando que uma inteligência artificial interprete o contexto da ocorrência e forneça informações relevantes sobre qual órgão ou serviço de emergência deve ser acionado. O sistema busca atuar como uma camada intermediária de apoio à decisão, reduzindo dúvidas relacionadas aos números emergenciais e auxiliando no direcionamento adequado da solicitação à entidade final.

A lógica do projeto atribui à inteligência artificial a responsabilidade de analisar a situação utilizando diferentes dados contextuais, como localização geográfica, data, horário e descrição da ocorrência. A partir dessas informações, o sistema pode interpretar características regionais, buscar notícias com informações relevantes, compreender o contexto da emergência e identificar quais serviços possuem maior compatibilidade com a situação relatada. Dessa forma, busca-se reduzir ambiguidades no acionamento dos serviços emergenciais e melhorar a acessibilidade ao atendimento adequado.

O fluxo geral de interação entre usuário, servidor e inteligência artificial pode ser visualizado na Figura 1.

Figura 1 – Fluxo de interação entre usuário, servidor e inteligência artificial



Elaborado pelo próprio autor (2026).

1.2.1 Objetivos Específicos

Com esse objetivo definido, este trabalho consiste em desenvolver uma aplicação móvel capaz de permitir o envio de relatos emergenciais em formato textual, sonoro e visual; implementar um servidor responsável pelo recebimento, processamento e gerenciamento das informações enviadas pelos usuários; integrar modelos de inteligência artificial para interpretação contextual das ocorrências relatadas; utilizar dados complementares, como localização geográfica, data e horário, para auxiliar na análise contextual das emergências; identificar e recomendar o órgão ou serviço de emergência mais adequado para cada situação reportada; reduzir ambiguidades relacionadas à diferenciação dos números de emergência existentes no Brasil; avaliar a viabilidade da utilização de inteligência artificial como ferramenta de apoio ao redirecionamento de solicitações emergenciais; e desenvolver uma prova de conceito funcional capaz de demonstrar o fluxo completo de comunicação entre usuário, servidor e sistema inteligente.

1.3 Correlatos

Para justificar a proposta apresentada anteriormente, torna-se necessário compreender o estado atual das tecnologias e sistemas relacionados ao atendimento emergencial. No cenário contemporâneo, é possível identificar tanto iniciativas privadas desenvolvidas com o objetivo de complementar limitações estruturais dos sistemas públicos de emergência, quanto modelos públicos integrados criados para centralizar e otimizar o acionamento de serviços emergenciais. Essas soluções buscam reduzir o tempo de resposta, melhorar a disponibilidade de informações contextuais e aumentar a eficiência na comunicação entre cidadãos e órgãos responsáveis pelo socorro.

1.3.1 RapidSOS

Representando a iniciativa privada, o RapidSOS consiste em uma plataforma voltada para integração tecnológica entre dispositivos móveis e centrais de atendimento emergencial. A solução foi desenvolvida com o objetivo de fornecer informações adicionais durante chamadas de emergência, permitindo que operadores tenham acesso a dados como localização, informações médicas e vídeos. A proposta da plataforma busca reduzir limitações em sistemas tradicionais de emergência, aumentando a disponibilidade de informações para as equipes responsáveis pelo atendimento e resposta às ocorrências (RAPIDSOS, 2025).

Apesar de ampliar a quantidade de dados disponíveis para os operadores, o RapidSOS não possui foco direto na interpretação automatizada da situação enfrentada pelo usuário através de inteligência artificial voltada ao redirecionamento contextual da ocorrência. Sua atuação está concentrada principalmente no compartilhamento de dados entre dispositivos e centrais emergenciais (RAPIDSOS, 2025).

A relação entre o RapidSOS e a proposta deste trabalho encontra-se principalmente na utilização de dados contextuais para auxiliar situações emergenciais. Ambos

utilizam informações contextuais como localização geográfica, horário e descrição da ocorrência para auxiliar no direcionamento adequado da solicitação emergencial.

1.3.2 911 e 112

Representando modelos públicos integrados de emergência, os sistemas 911 e 112 constituem alguns dos principais exemplos de números universais de emergência utilizados internacionalmente. O número 911 é amplamente utilizado nos Estados Unidos e Canadá como canal centralizado para acionamento de polícia, bombeiros e serviços médicos emergenciais, enquanto o 112 foi adotado como número padrão de emergência em diversos países da União Europeia (ASSOCIATION, 2024a; ASSOCIATION, 2024b).

Esses modelos possuem como principal característica a centralização do acesso aos serviços emergenciais através de um único número telefônico, reduzindo ambiguidades relacionadas à identificação do órgão responsável pela ocorrência. Além disso, sistemas modernos baseados em 911 e 112 passaram a incorporar recursos tecnológicos complementares, como compartilhamento automático de localização, integração digital entre centrais, suporte a comunicação multimodal e encaminhamento inteligente de chamadas (ASSOCIATION, 2024a; ASSOCIATION, 2024b).

Embora apresentem elevado nível de integração estrutural, esses sistemas dependem diretamente da infraestrutura pública de emergência existente em cada país, além de exigirem elevado nível de padronização tecnológica e administrativa para seu funcionamento eficiente. A relação entre esses sistemas e a proposta deste trabalho encontra-se principalmente na tentativa de centralizar e simplificar o acesso aos serviços emergenciais, reduzindo dificuldades relacionadas à identificação do órgão responsável pela ocorrência. Entretanto, diferentemente dos modelos tradicionais de 911 e 112, o projeto Centro de Ajuda propõe a utilização de inteligência artificial para interpretar contextualmente a situação relatada pelo usuário antes do direcionamento emergencial.

2 Revisão Bibliográfica

Para desenvolver um aplicativo capaz de utilizar inteligência artificial para interpretar situações emergenciais, compreender diferentes tipos de entrada de dados e auxiliar no direcionamento contextual de solicitações, torna-se necessário compreender os principais conceitos e tecnologias relacionados à área. Esta seção apresenta os fundamentos teóricos que sustentam o desenvolvimento da proposta, abordando conceitos de inteligência artificial, aprendizado de máquina, redes neurais, modelos de linguagem e processamento multimodal.

2.1 *Inteligência Artificial*

Segundo Russell e Norvig (2020), a inteligência artificial corresponde a uma área da computação voltada ao desenvolvimento de sistemas capazes de executar tarefas que normalmente exigiriam capacidades cognitivas humanas, como reconhecimento de padrões, interpretação de linguagem, tomada de decisão e resolução de problemas. Segundo sua abordagem, a inteligência artificial pode ser definida como o estudo de agentes inteligentes, isto é, sistemas capazes de perceber o ambiente ao seu redor e realizar ações com o objetivo de maximizar suas chances de sucesso em determinada tarefa.

2.1.1 Aprendizado de Máquina

A principal base para a inteligência artificial moderna é o aprendizado de máquina, também conhecido como *machine learning*. Essa área consiste no desenvolvimento de modelos computacionais capazes de aprender padrões e relações a partir de dados, permitindo que sistemas realizem previsões, classificações e tomadas de decisão sem depender exclusivamente de regras programadas manualmente.

Segundo Mitchell (1997), um programa é capaz de aprender a partir da experiência obtida durante a execução de uma tarefa, melhorando seu desempenho conforme

novos dados são analisados. Essa definição estabelece o aprendizado de máquina como um processo baseado em adaptação contínua, no qual os sistemas ajustam seus parâmetros internos de acordo com os padrões identificados durante o treinamento e utilização do modelo.

Esse conceito tornou-se um dos principais pilares da inteligência artificial devido à sua capacidade de identificar padrões complexos em grandes volumes de dados. Em vez de depender apenas de instruções fixas, modelos de aprendizado de máquina conseguem construir representações estatísticas capazes de reconhecer relações entre diferentes variáveis, permitindo aplicações em reconhecimento de voz, visão computacional, sistemas de recomendação, detecção de fraudes e processamento de linguagem natural (IBM, 2024).

De modo geral, os métodos de aprendizado de máquina podem ser divididos em três principais categorias: aprendizado supervisionado, aprendizado não supervisionado e aprendizado por reforço. No aprendizado supervisionado, o modelo é treinado utilizando exemplos rotulados, nos quais já existe uma resposta esperada associada aos dados de entrada. Essa abordagem é amplamente utilizada em tarefas de classificação e previsão. No aprendizado não supervisionado, os dados não possuem rótulos prévios, cabendo ao sistema identificar padrões, agrupamentos e relações ocultas nos dados analisados. Já o aprendizado por reforço consiste em um processo no qual agentes computacionais aprendem por tentativa e erro a partir de recompensas e penalidades recebidas durante interações com um ambiente (IBM, 2024).

2.1.2 Redes Neurais e Deep Learning

Entre as principais abordagens utilizadas no aprendizado de máquina moderno, destacam-se as redes neurais artificiais e as técnicas de *deep learning*. Essas estruturas tornaram-se responsáveis por grande parte dos avanços recentes da inteligência artificial,

especialmente em tarefas relacionadas ao reconhecimento de padrões complexos, interpretação de linguagem natural, visão computacional e processamento de áudio.

As redes neurais artificiais consistem em modelos computacionais inspirados no funcionamento biológico do cérebro humano. Sua estrutura é composta por neurônios artificiais interconectados, organizados em camadas responsáveis por receber, processar e transmitir informações. Durante o treinamento, os pesos dessas conexões são ajustados matematicamente com o objetivo de minimizar erros e melhorar a capacidade do modelo em reconhecer padrões presentes nos dados analisados (GOODFELLOW; BENGIO; COURVILLE, 2016).

Segundo Goodfellow, Bengio e Courville (2016), uma das principais características das redes neurais é sua capacidade de aprender representações hierárquicas dos dados. Isso significa que diferentes camadas da rede conseguem identificar níveis progressivamente mais complexos de informação. Em reconhecimento de imagens, por exemplo, camadas iniciais podem identificar linhas e bordas simples, enquanto camadas mais profundas passam a reconhecer formas, objetos e relações contextuais completas.

Dessa forma, o termo *deep learning*, também conhecido como aprendizado profundo, refere-se à utilização de redes neurais compostas por múltiplas camadas ocultas. O aumento da profundidade dessas arquiteturas permitiu avanços significativos no desempenho de modelos de inteligência artificial, especialmente em aplicações envolvendo grandes volumes de dados. Diferentemente de métodos tradicionais, nos quais características relevantes precisavam ser definidas manualmente, modelos de aprendizado profundo conseguem aprender automaticamente representações internas diretamente a partir dos dados utilizados durante o treinamento.

Eventualmente, diferentes arquiteturas de redes neurais passaram a ser desenvolvidas para resolver problemas específicos. Entre os principais exemplos estão as redes neurais convolucionais (Convolutional Neural Networks (CNNs)) amplamente utilizadas em tarefas de visão computacional e reconhecimento de imagens, e as redes neurais

recorrentes (Recurrent Neural Networks (RNNS)) desenvolvidas para lidar com dados sequenciais, como textos e traduções automáticas (GOODFELLOW; BENGIO; COURVILLE, 2016).

Segundo Goodfellow, Bengio e Courville (2016), as redes neurais recorrentes possuem a capacidade de manter informações relacionadas a estados anteriores durante o processamento da sequência, permitindo que o modelo considere o contexto de elementos previamente analisados. Essa característica tornou as RNNs uma das principais abordagens utilizadas nas primeiras aplicações modernas de processamento de linguagem natural, reconhecimento de fala e tradução automática.

Apesar de representarem um avanço significativo em relação a modelos tradicionais, arquiteturas recorrentes apresentavam limitações relacionadas à dificuldade de processamento paralelo e à perda de informações em sequências muito longas, problema conhecido como *vanishing gradient*. Essas limitações impulsionaram o desenvolvimento de arquiteturas mais eficientes para interpretação contextual e processamento de grandes volumes de dados sequenciais (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.1.3 Arquitetura Transformer

Diferente de arquiteturas anteriores, como RNNS, Transformer foi desenvolvido com o objetivo de processar informações de maneira paralela, reduzindo limitações relacionadas à dependência sequencial dos dados. Enquanto modelos recorrentes analisavam palavras e informações uma após a outra, a arquitetura Transformer passou a processar sequências inteiras simultaneamente, aumentando significativamente a eficiência computacional e a capacidade de treinamento em grandes conjuntos de dados (VASWANI *et al.*, 2017).

O principal componente responsável pelo funcionamento desta arquitetura é o mecanismo de atenção, especialmente o chamado *self-attention*. Esse mecanismo permite que o modelo avalie a relevância de diferentes elementos de uma sequência em relação aos

demais elementos presentes no mesmo contexto. Dessa forma, o Transformer consegue compreender relações semânticas entre palavras, identificar dependências contextuais distantes e interpretar significados de maneira mais consistente (VASWANI *et al.*, 2017).

Segundo Vaswani *et al.* (2017), Outra vantagem substancial da arquitetura Transformer é sua escalabilidade. O modelo demonstrou elevada capacidade de desempenho conforme aumentavam o volume de dados utilizados durante o treinamento e a quantidade de parâmetros presentes na rede neural. Essa característica contribuiu diretamente para o desenvolvimento de modelos modernos capazes de realizar tarefas complexas de interpretação contextual, geração de texto e raciocínio baseado em linguagem natural.

2.2 LLMs

A elevada escalabilidade da arquitetura Transformer possibilitou o desenvolvimento dos chamados modelos de linguagem de grande porte (Large Language Models (LLMs)). Esses modelos consistem em redes neurais treinadas em enormes volumes de dados textuais com o objetivo de identificar padrões estatísticos, relações semânticas e estruturas linguísticas presentes na linguagem humana..

Segundo Brown *et al.* (2020), os LLMs são capazes de aprender relações contextuais complexas entre palavras, frases e conceitos por meio do treinamento em grandes conjuntos de dados. Durante esse processo, os textos são divididos em tokens, que representam palavras completas ou fragmentos de palavras. Esses tokens são convertidos em representações numéricas de alta dimensão, permitindo que o modelo analise matematicamente relações de proximidade e contexto entre diferentes elementos linguísticos.

Com base nessas relações estatísticas, os modelos conseguem calcular a probabilidade de um token suceder outro dentro de uma sequência textual. Esse processo permite que LLMs realizem tarefas como geração de texto, tradução automática, sumarização, interpretação contextual e respostas baseadas em linguagem natural. Quanto maior o

volume de dados e a quantidade de parâmetros utilizados durante o treinamento, maior tende a ser a capacidade do modelo em compreender contextos complexos e produzir respostas coerentes (BROWN *et al.*, 2020).

2.2.1 Engenharia de Prompt

Para se comunicar com LLMS, é necessário fornecer instruções textuais conhecidas como prompts. Segundo Openai (2024), prompts consistem em instruções textuais utilizadas para orientar o comportamento do modelo durante a geração de respostas. Essas instruções podem incluir contexto adicional, regras de comportamento, exemplos de entrada e saída, restrições de formato e descrições detalhadas sobre a tarefa desejada. Quanto mais claro e contextualizado for o prompt, maior tende a ser a capacidade do modelo em produzir resultados coerentes com o objetivo esperado.

Essa interpretação contextual desempenha papel fundamental no funcionamento dos LLMs. Palavras isoladas podem possuir múltiplos significados dependendo do contexto em que aparecem. O termo banco, por exemplo, pode se referir tanto a uma instituição financeira quanto a um objeto utilizado para sentar. Nesse cenário, os modelos utilizam relações estatísticas aprendidas durante o treinamento para analisar palavras vizinhas, estruturas semânticas e padrões linguísticos capazes de indicar qual significado possui maior probabilidade dentro daquele contexto específico (OPENAI, 2024). Dessa forma, a maneira como as informações são apresentadas no prompt influencia diretamente a interpretação realizada pelo modelo.

Para orientar essa interpretação, diferentes técnicas de engenharia de prompt podem ser empregadas. Entre elas estão o *zero-shot*, no qual o modelo executa uma tarefa apenas com base na instrução fornecida; o *few-shot* que apresenta exemplos de entrada e saída para demonstrar o comportamento esperado; e o *chain-of-thought* que orienta a decomposição de problemas complexos em etapas intermediárias de raciocínio. Por exemplo, para classificar uma emergência, o prompt pode apenas solicitar a classifi-

cação (*zero-shot*), apresentar relatos previamente classificados (*few-shot*) ou solicitar que os elementos da ocorrência sejam analisados antes da resposta final (*chain-of-thought*) (WEI *et al.*, 2022).

Embora essas técnicas contribuam para orientar o modelo e tornar suas respostas mais previsíveis, elas não eliminam as limitações inerentes aos LLMs. Pequenas alterações na estrutura textual da instrução podem gerar respostas significativamente diferentes, especialmente em tarefas complexas ou ambíguas. Além disso, modelos de linguagem podem produzir informações incorretas, fenômeno frequentemente chamado de *hallucination* no qual o sistema gera conteúdos sintaticamente coerentes, porém factualmente incorretos ou inexistentes (OPENAI, 2024).

Devido a essas características, a construção de prompts tornou-se uma etapa essencial no desenvolvimento de aplicações baseadas em inteligência artificial generativa. Estratégias como definição explícita de contexto, utilização de exemplos, delimitação de regras e restrição de formato são frequentemente utilizadas para aumentar a previsibilidade das respostas produzidas pelos modelos (OPENAI, 2024).

2.3 Modelos Multimodais

A evolução dos modelos de linguagem de grande porte também possibilitou o desenvolvimento dos chamados modelos multimodais, sistemas de inteligência artificial capazes de processar e interpretar múltiplos tipos de dados simultaneamente. Diferentemente de modelos tradicionais voltados exclusivamente para uma única modalidade de entrada, esses sistemas são treinados utilizando diferentes formas de informação, como linguagem natural, imagens, áudio e vídeo (OPENAI, 2023).

Tais modelos possuem a capacidade de integrar diferentes modalidades em uma mesma representação contextual, permitindo compreender relações entre informações textuais, visuais e sonoras presentes nos dados analisados. Como resultado, essa abordagem amplia significativamente a capacidade interpretativa da inteligência artificial,

tornando possível a execução de tarefas mais complexas que envolvem múltiplos formatos de informação (OPENAI, 2023).

Segundo Openai (2023), o funcionamento desses modelos geralmente ocorre através da conversão das diferentes modalidades em representações numéricas compatíveis dentro do espaço vetorial utilizado pela rede neural. Dessa forma, textos, imagens, sons e vídeos podem ser processados conjuntamente pelo modelo, permitindo que relações contextuais entre diferentes tipos de informação sejam identificadas durante a inferência.

A utilização de arquiteturas multimodais tornou-se especialmente relevante em aplicações modernas relacionadas à análise de imagens, reconhecimento de fala, interpretação audiovisual e assistentes inteligentes capazes de compreender diferentes formatos de entrada simultaneamente. Além disso, esses modelos passaram a demonstrar capacidade de realizar tarefas cruzadas entre modalidades, como descrever imagens em linguagem natural, responder perguntas sobre conteúdos visuais e interpretar informações sonoras contextualizadas (OPENAI, 2023).

2.3.1 Processamento de Imagem

Diferente de LLMs tradicionais voltadas exclusivamente para linguagem natural, os modelos multimodais possuem a capacidade de interpretar conteúdos visuais e relacioná-los ao contexto textual fornecido pelo usuário. Essa capacidade ampliou significativamente o potencial de aplicações baseadas em inteligência artificial, especialmente em cenários nos quais informações visuais desempenham papel importante para compreensão da situação analisada. Modelos multimodais modernos conseguem identificar objetos, interpretar cenas, reconhecer padrões visuais e responder perguntas relacionadas ao conteúdo presente em imagens, permitindo que informações textuais e visuais sejam analisadas conjuntamente durante a inferência (OPENAI, 2023).

Segundo Openai (2023), o funcionamento do processamento visual nesses modelos ocorre através da conversão dos elementos presentes na imagem em representações

numéricas compatíveis com o espaço vetorial utilizado pela rede neural. Durante esse processo, características como formas, cores, texturas, objetos e relações espaciais são transformadas em *embeddings* (representações vetoriais que preservam informações semânticas e estruturais dos dados) capazes de serem interpretados juntamente com dados textuais e contextuais. Dessa forma, o modelo consegue estabelecer relações semânticas entre diferentes modalidades de entrada, permitindo interpretação contextual integrada entre imagem e linguagem natural.

O processamento de imagens em modelos multimodais possui forte relação com técnicas de visão computacional RNNS anteriormente citados no documento. Segundo Goodfellow, Bengio e Courville (2016), arquiteturas convolucionais foram desenvolvidas especificamente para identificar padrões visuais em imagens através da aplicação sucessiva de filtros matemáticos capazes de reconhecer características progressivamente mais complexas. Em camadas iniciais, o modelo pode identificar bordas, linhas e texturas simples, enquanto camadas mais profundas conseguem reconhecer objetos completos, estruturas e relações contextuais presentes na cena analisada.

Embora modelos multimodais modernos não dependam exclusivamente de CNNs tradicionais, muitas arquiteturas atuais ainda utilizam princípios derivados do aprendizado profundo aplicado à visão computacional para interpretar informações visuais. Em sistemas baseados em Transformers multimodais, por exemplo, imagens podem ser divididas em pequenas regiões chamadas *patches*, que são convertidas em tokens visuais semelhantes aos tokens textuais utilizados em modelos de linguagem natural. Dessa forma, o modelo consegue aplicar mecanismos de atenção contextual também sobre elementos visuais da imagem (DOSOVITSKIY *et al.*, 2021).

2.3.2 Processamento de Áudio

O processamento de áudio em modelos multimodais permite que sistemas interpretem informações sonoras e relacionem esses dados ao contexto textual e visual.

Segundo Radford *et al.* (2022), sistemas modernos de processamento de áudio convertem sinais sonoros em representações numéricas adequadas ao processamento por redes neurais.

Grande parte das aplicações contemporâneas utiliza técnicas de reconhecimento automático de fala (*Automatic Speech Recognition – ASR*) (RADFORD *et al.*, 2022).

As primeiras aplicações modernas de reconhecimento de fala utilizaram fortemente arquiteturas recorrentes (GOODFELLOW; BENGIO; COURVILLE, 2016).

2.3.3 Processamento de Vídeo

O processamento de vídeo em modelos multimodais envolve a análise simultânea de informações visuais e temporais (OPENAI, 2023). Arquiteturas modernas baseadas em Transformers multimodais demonstraram elevado desempenho nesse domínio (VASWANI *et al.*, 2017).

3 Metodologia

A metodologia utilizada neste trabalho consiste na análise de requisitos para o desenvolvimento de uma prova de conceito funcional de um sistema inteligente voltado ao auxílio no redirecionamento de solicitações emergenciais. O sistema foi estruturado utilizando arquitetura cliente-servidor.

3.1 Critérios de Escolha para o(s) Modelo(s) LLM

O funcionamento da proposta depende diretamente da integração entre o servidor backend e modelos de inteligência artificial. Foram definidos critérios técnicos: capacidade multimodal (texto, áudio, imagem), disponibilidade em nuvem e capacidade de pesquisa na internet.

Entre os modelos analisados destacam-se o GPT-5.5, Claude Opus 4.8, Gemini 3.5 Flash, Llama 4 Maverick, Qwen 3.6 Plus, Nemotron 3 Nano Omni 30B e North Mini Code. As informações correspondem à análise realizada em 10 de junho de 2026.

A partir dos critérios estabelecidos, foi elaborada uma comparação preliminar entre os modelos selecionados.

Tabela 1 – Comparação de modelos quanto aos critérios técnicos

Critério	GPT	Claude	Llama	Qwen	Gemini	Nemo- tron	North
Multi-modal	Parcial	Parcial	Parcial	Parcial	Sim	Parcial	Não
Nuvem	Sim	Sim	Sim	Sim	Sim	Sim	Sim
Pesquisa	Sim	Sim	Sim	Sim	Sim	Não	Não

Elaborado pelo próprio autor (2026).

3.1.1 GPT-5.5

O GPT-5.5, desenvolvido pela OpenAI, foi selecionado para análise (OPENAI, 2026). Em relação à multimodalidade, o GPT-5.5 oferece suporte ao processamento de texto, imagens e áudio por meio da plataforma da OpenAI, porém sem envio simultâneo

nativo das três modalidades em uma única solicitação, atendendo parcialmente ao critério (OPENAI, 2026). É disponibilizado via APIs em nuvem (OPENAI, 2026) e oferece suporte à pesquisa na internet (OPENAI, 2026).

3.1.2 Claude Opus 4.8

Desenvolvido pela Anthropic, o Claude Opus 4.8 aceita texto e imagens, mas não áudio nativo, atendendo parcialmente ao critério (ANTHROPIC, 2026). É disponibilizado via APIs em nuvem (ANTHROPIC, 2026) e disponibiliza recursos de pesquisa na internet (ANTHROPIC, 2026).

3.1.3 Llama 4 Maverick

O Llama 4 Maverick, desenvolvido pela Meta, oferece suporte a texto e imagens, sem áudio nativo (META, 2026a). É disponibilizado em nuvem (META, 2026b) e pode ser integrado a ferramentas externas de busca (META, 2026a).

3.1.4 Qwen 3.6 Plus

O Qwen 3.6 Plus, desenvolvido pela Alibaba Cloud, disponibiliza texto e imagens, sem áudio nativo (CLOUD, 2026a). Foi projetado para correlacionar texto e imagens (CLOUD, 2026b). É acessível via nuvem (CLOUD, 2026a) e permite integração com busca externa (CLOUD, 2026a).

3.1.5 Gemini 3.5 Flash

O Gemini 3.5 Flash, desenvolvido pela Google, oferece suporte nativo a texto, imagens, áudio, vídeo e documentos, atendendo integralmente ao critério (GOOGLE, 2026a). É disponibilizado via APIs em nuvem (GOOGLE, 2026b) e disponibiliza pesquisa nativa na internet (GOOGLE, 2026a).

3.1.6 Nemotron 3 Nano Omni

O Nemotron 3 Nano Omni 30B, desenvolvido pela NVIDIA, aceita texto, imagens, áudio e vídeo (NVIDIA, 2026). Foi projetado para raciocínio conjunto multimodal (NVIDIA, 2026). É disponibilizado via NVIDIA NIM em nuvem (NVIDIA, 2026) e não possui busca nativa, mas permite integração externa (NVIDIA, 2026).

3.1.7 North Mini Code

O North Mini Code, desenvolvido pela Cohere, foi desenvolvido essencialmente para texto e código, sem suporte nativo a imagens ou áudio (COHERE, 2026). Não atende ao requisito multimodal (COHERE, 2026). É disponibilizado sob licença Apache 2.0 em nuvem (COHERE, 2026) e não possui busca nativa (COHERE, 2026).

3.2 *Requisitos do Frontend*

O frontend corresponde à camada responsável pela interação direta com o usuário. Foram identificadas três áreas fundamentais: interface do usuário, compatibilidade multiplataforma e comunicação cliente-servidor.

3.2.1 Interface do Usuário

A construção da interface foi realizada utilizando HTML, CSS e JavaScript (MOZILLA, 2024a). O HTML foi empregado para estruturar semanticamente a interface, o CSS para apresentação e o JavaScript para lógica e comunicação com o backend.

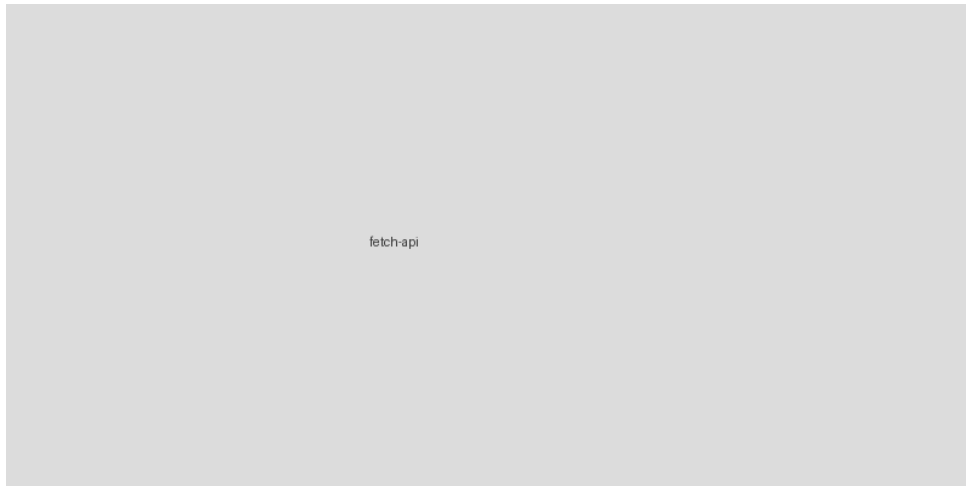
3.2.2 Compatibilidade Multiplataforma

A compatibilidade multiplataforma é proporcionada pela utilização de tecnologias web. Além disso, a aplicação pode ser disponibilizada como uma *Progressive Web App* (PWA) (MOZILLA, 2024b).

3.2.3 Comunicação Cliente-Servidor

A comunicação entre frontend e backend foi implementada utilizando a API Fetch (MOZILLA, 2024c).

Figura 2 – Fluxo da API Fetch com Promise



Elaborado pelo próprio autor (2026).

3.3 Requisitos do Backend

O backend concentra o processamento necessário para transformar os dados enviados pelo usuário em resposta utilizável. Essa camada cumpre quatro funções: receber solicitações multimodais, complementar com localização, encaminhar ao LLM e devolver resposta estruturada.

3.3.1 Infraestrutura Backend

A infraestrutura foi construída como aplicação web organizada em torno de uma rota principal de atendimento. Foram utilizados Node.js, Express e Fetch (FOUNDATION, 2024a; FOUNDATION, 2024b; MOZILLA, 2024c).

3.3.2 Ambiente de Execução

O backend foi implementado em Node.js (FOUNDATION, 2024a). O modelo assíncrono do Node.js foi especialmente relevante para aguardar serviços externos sem bloquear a execução.

3.3.3 Estrutura da Aplicação

Sobre o ambiente Node.js, o Express foi utilizado para criar a interface HTTP (FOUNDATION, 2024b). Quando uma requisição é recebida, o Express direciona os dados para a função responsável e verifica coordenadas geográficas.

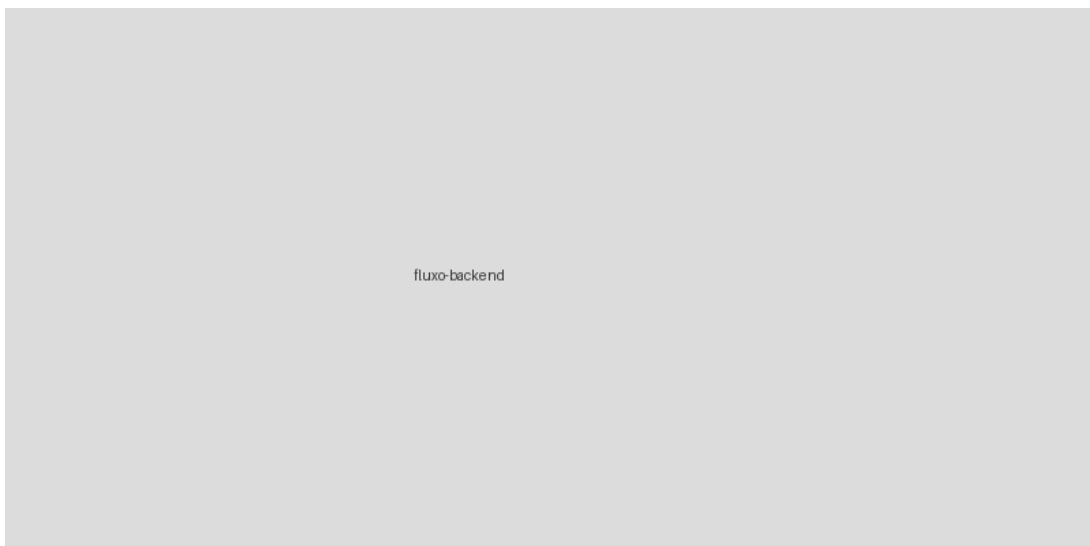
3.3.4 Integração com Serviços Externos

As chamadas foram implementadas com a API Fetch (MOZILLA, 2024c). Uma integração ocorre com o serviço de geocodificação reversa do Google Maps e outra com a plataforma de distribuição de IAs.

3.4 Fluxo de Comunicação

O fluxo inicia-se quando o frontend envia relato em texto, áudio ou imagem com coordenadas. O backend realiza geocodificação reversa, compõe contexto com horário e relato, envia à LLM e retorna serviço indicado, número e resumo. A sequência completa pode ser observada na Figura 3.

Figura 3 – Fluxo de comunicação entre frontend, backend e APIs externas



Elaborado pelo próprio autor (2026).

4 Desenvolvimento

Neste capítulo são apresentados os detalhes de desenvolvimento do sistema Centro de Ajuda.

4.1 Abordagem Geral

O desenvolvimento foi dividido em duas camadas: aplicação cliente e aplicação servidora. O foco concentrou-se na implementação do backend e na validação do fluxo completo. O cliente móvel foi representado por protótipo funcional no Figma.

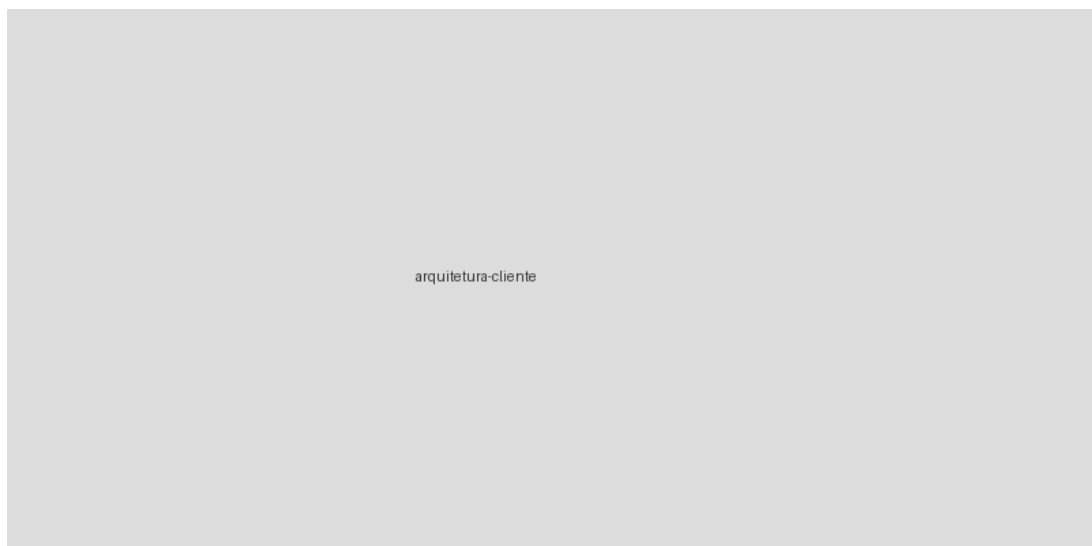
4.2 Implementação da Aplicação Cliente

A aplicação cliente foi projetada para dispositivos móveis e organizada em módulos responsáveis pela interface, captura de entradas e envio ao servidor.

4.2.1 Arquitetura do Cliente

A arquitetura foi organizada de acordo com o fluxo de interação, abrangendo coleta, preparação, comunicação e apresentação do resultado, conforme Figura 4.

Figura 4 – Arquitetura da aplicação cliente



Elaborado pelo próprio autor (2026).

4.2.2 Tecnologias da Aplicação Cliente

A interface foi desenvolvida com HTML5, CSS3 e JavaScript, com Bootstrap para responsividade. A comunicação utiliza Fetch API. A aplicação foi implementada como PWA básica com `manifest.json` e Service Worker.

4.2.3 Tela Inicial

A tela inicial concentra todas as funcionalidades para registro da ocorrência, conforme Figura 5.

Figura 5 – Tela inicial do Centro de Ajuda



Elaborado pelo próprio autor (2026).

Captura de texto

A entrada textual foi implementada via `<textarea>` com limite de 2000 caracteres.

Código 1 – Campo de texto da ocorrência

```
occurrence.html
1 <textarea id="occurrence-text" name="text" rows="5" maxlength="2000"
  placeholder="Descreva sua ocorrência..."></textarea>
```

Elaborado pelo próprio autor (2026).

Código 2 – Captura de texto via DOM

capture.js

```
1 function captureText() {  
2   const textInput = document.getElementById("occurrence-text");  
3   const text = textInput.value.trim();  
4   return text || null;  
5 }
```

*Elaborado pelo próprio autor (2026).***Captura de áudio**

A captura de áudio foi implementada com MediaDevices API e MediaRecorder API.

Código 3 – Solicitação de acesso ao microfone

audio.js

```
1 let audioStream = null;  
2 async function requestMicrophoneAccess() {  
3   try {  
4     audioStream = await navigator.mediaDevices.getUserMedia({ audio:  
       true });  
5     return audioStream;  
6   } catch (error) {  
7     console.error("Não foi possível acessar o microfone:", error);  
8     return null;  
9   }  
10 }
```

Elaborado pelo próprio autor (2026).

Código 4 – Gravação de áudio com MediaRecorder

```
recorder.js
1  let mediaRecorder = null;
2  let audioChunks = [];
3  let audioBlob = null;
4  async function startAudioRecording() {
5    const stream = await requestMicrophoneAccess();
6    if (!stream) return false;
7    audioChunks = [];
8    audioBlob = null;
9    mediaRecorder = new MediaRecorder(stream);
10   mediaRecorder.addEventListener("dataavailable", event => {
11     if (event.data.size > 0) audioChunks.push(event.data);
12   });
13   mediaRecorder.addEventListener("stop", () => {
14     audioBlob = new Blob(audioChunks, { type: mediaRecorder.mimeType });
15     stream.getTracks().forEach(track => track.stop());
16     audioStream = null;
17   });
18   mediaRecorder.start();
19   return true;
20 }
21 function stopAudioRecording() {
22   if (mediaRecorder?.state === "recording") mediaRecorder.stop();
23 }
```

Elaborado pelo próprio autor (2026).

Captura de imagens

Campo de seleção com `capture="environment"` e preview via FileReader.

Código 5 – Captura de imagem

```
image.js
1  const imageInput = document.getElementById("image-input");
2  const imagePreview = document.getElementById("image-preview");
3  let imageFile = null;
4  imageInput.addEventListener("change", event => {
5    const selectedFile = event.target.files[0];
6    if (!selectedFile || !selectedFile.type.startsWith("image/")) return;
7    imageFile = selectedFile;
8    const reader = new FileReader();
9    reader.addEventListener("load", () => {
10      imagePreview.src = reader.result;
11      imagePreview.hidden = false;
12    });
13    reader.readAsDataURL(imageFile);
14  });
```

Elaborado pelo próprio autor (2026).

Envio da solicitação

Código 6 – Envio da ocorrência ao servidor

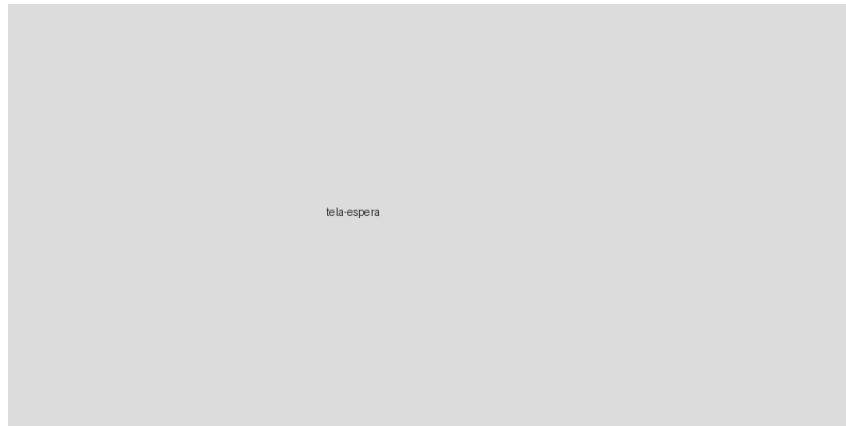
```
submit.js
1  const helpForm = document.getElementById("help-form");
2  const submitButton = document.getElementById("submit-button");
3  let requestInProgress = false;
4  helpForm.addEventListener("submit", async event => {
5    event.preventDefault();
6    if (requestInProgress) return;
7    const text = captureText();
8    const hasAudio = audioBlob instanceof Blob;
9    const hasImage = imageFile instanceof File;
10   if (!text && !hasAudio && !hasImage) {
11     alert("Forneça um relato por texto, áudio ou imagem.");
12     return;
13   }
14   requestInProgress = true;
15   submitButton.disabled = true;
16   showScreen("waiting-screen");
17   try {
18     const location = await getCurrentLocation();
19     const occurrenceData = await buildOccurrenceData(text, location);
20     const response = await fetch("/help", {
21       method: "POST",
22       headers: { "Content-Type": "application/json" },
23       body: JSON.stringify(occurrenceData)
24     });
25     if (!response.ok) throw new Error(`Erro HTTP: ${response.status}`);
26     await handleServerResponse(response);
27   } catch (error) {
28     console.error("Erro ao enviar a solicitação:", error);
29     showResponseError("Não foi possível processar a solicitação. Tente novamente.");
30   } finally {
31     requestInProgress = false;
32     submitButton.disabled = false;
33   }
34 });
```

Elaborado pelo próprio autor (2026).

4.2.4 Tela de Espera

Após o envio, a aplicação apresenta tela de espera com animação de carregamento até retorno do servidor, conforme Figura 6.

Figura 6 – Tela de espera

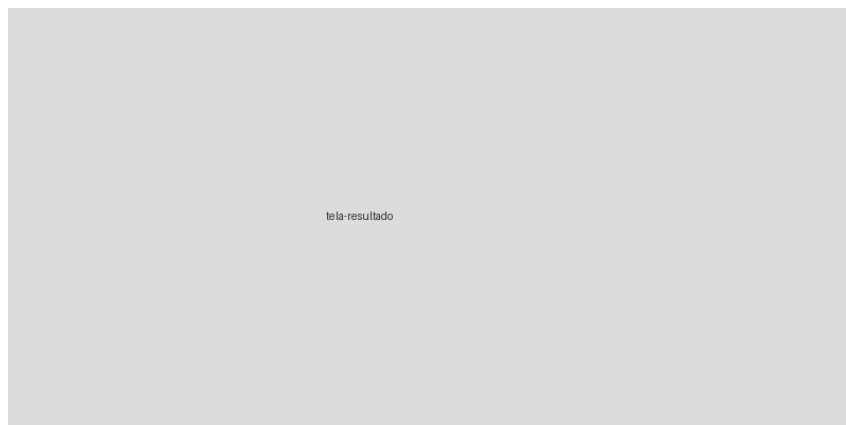


Elaborado pelo próprio autor (2026).

4.2.5 Tela de Resposta

A tela exibe serviço recomendado, telefone e resumo do contexto, conforme Figura 7.

Figura 7 – Tela de resposta com recomendação



Elaborado pelo próprio autor (2026).

Código 7 – Tratamento da resposta do servidor

```
handle.js
1  async function handleServerResponse(response) {
2    let result;
3    try {
4      result = await response.json();
5    } catch (error) {
6      showResponseError("O servidor retornou uma resposta inválida. Tente novamente.");
7      return;
8    }

    const validResponse = response.ok && typeof result?.service_name ===
9      "string" && typeof result?.phone_number === "string" && typeof
      result?.emergency_context === "string";
10   if (!validResponse) {
11     showResponseError(result?.error || "Não foi possível analisar a ocorrência. Tente novamente.");
12     return;
13   }

14   document.getElementById("service-name").textContent =
      result.service_name;
15   document.getElementById("phone-number").textContent =
      result.phone_number;
16   document.getElementById("emergency-context").textContent =
      result.emergency_context;
17   configureCallButton(result.phone_number);
18   showScreen("result-screen");
19 }

20 function showResponseError(message) {
21   const errorMessage = document.getElementById("request-error");
22   errorMessage.textContent = message;
23   errorMessage.hidden = false;
24   showScreen("initial-screen");
25 }
```

Elaborado pelo próprio autor (2026).

Código 8 – Redirecionamento para ligação

```
call.js
1  function configureCallButton(phoneNumber) {
2    const callButton = document.getElementById("call-button");
3    const sanitizedNumber = phoneNumber.replace(/\D/g, "");
4    if (!sanitizedNumber) {
5      callButton.hidden = true;
6      callButton.removeAttribute("href");
7      return;
8    }
9    callButton.href = `tel:${sanitizedNumber}`;
10   callButton.hidden = false;
11 }
```

Elaborado pelo próprio autor (2026).

4.3 Implementação do Servidor

O servidor foi estruturado de forma modular (inicialização, rotas, processamento, serviços externos). Inicialização na porta 27020 com Express e CORS.

Código 9 – Inicialização do servidor

```
main.js
1  import express from 'express';
2  const app = express();
3  app.use(express.json());
4
5  app.get('/api/status', (req, res) => {
6    res.json({ status: 'online', campus: 'IFBA-SAJ' });
7  });
8
9  app.post('/api/login', (req, res) => {
10   const { user, password } = req.body;
11   // Autenticação simulada
12   res.json({ ok: user === 'admin' && password === 'admin' });
13 });
14
15 const PORT = process.env.PORT || 3000;
16 app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

Elaborado pelo próprio autor (2026).

Código 10 – Configuração Express com CORS e bodyParser

```
express.js
1  const express = require("express");
2  const cors = require("cors");
3  const bodyParser = require("body-parser");
4  const app = express();
5  app.use(bodyParser.json({ limit: "10mb" }));
6  app.use(bodyParser.urlencoded({ extended: true }));
7  app.use(cors({
8    origin: "*",
9    methods: ["GET", "POST", "OPTIONS"],
10   allowedHeaders: ["Content-Type", "Authorization"],
11   credentials: false,
12   maxAge: 86400
13 }));
14 async function startExpress(port) {
15   setupEvents();
16   return new Promise((resolve, reject) => {
17     app.listen(port, "0.0.0.0", error => {
18       if (error) return reject(error);
19       console.log(`Express: Listening on ${port}`);
20       resolve();
21     });
22   });
23 }
24 module.exports = { startExpress, app };
```

Elaborado pelo próprio autor (2026).

Código 11 – Geocodificação reversa

```
geocode.js
1  async function getLocation(lat, lon) {
2    const url = "https://maps.googleapis.com/maps/api/geocode/json" + `?
    latlng=${lat},${lon}` + `&key=${process.env.GOOGLEKEY}`;
3    const response = await fetch(url);
4    const data = await response.json();
5    return extractLocation(data);
6  }
7  function extractLocation(data) {
8    const components = data.results[0].address_components;
9    let street = null, neighborhood = null, city = null, state = null,
    country = null, zip = null;
10   for (const component of components) {
11     const types = component.types;
12     if (types.includes("route")) street = component.long_name;
13     if (types.includes("sublocality") ||
        types.includes("sublocality_level_1") ||
        types.includes("neighborhood")) neighborhood = component.long_name;
14     if (types.includes("administrative_area_level_2")) city =
        component.long_name;
15     if (types.includes("administrative_area_level_1")) state =
        component.long_name;
16     if (types.includes("country")) country = component.long_name;
17     if (types.includes("postal_code")) zip = component.long_name;
18   }
19   return { street, neighborhood, city, state, country, zip };
20 }
```

Elaborado pelo próprio autor (2026).

Endpoints carregados dinamicamente do diretório `endpoints/` (ex.: `help.js` → `/help`). Comunicação com Google Reverse Geocoding e plataforma de IAs via Fetch, com conversão Base64 para multimodal.

4.4 Construção do Prompt

Antes da comunicação com o LLM, o servidor constrói três componentes: System Prompt (regras, formato JSON), Assistant Prompt (contexto dinâmico: horário, endereço via `assistantprompt(context)`) e mensagem do usuário (texto + imagem Base64

+ áudio Base64). Essa separação reduz acoplamento e permite alterar regras sem afetar contexto.

Código 12 – System Prompt

```
systemprompt.js

const systemprompt = ` # Função Você é um classificador de ocorrências.
Analise todas as informações disponíveis e determine qual serviço deve ser
contatado. # Processo 1. Analise todas as evidências. 2. Se não for
emergência oficial, pesquise serviços compatíveis. 3. Considere
1 localização, horário e área de atendimento. 4. Escolha o serviço mais
apropriado. # Regras - Considere texto, áudio, imagem, localização e
horário em conjunto. - Priorize serviços oficiais. - Escolha apenas um
serviço. - Não forneça diagnósticos. - Não invente telefones. # Saída
Retorne somente JSON válido: { "service_name": "Nome", "phone_number":
"Telefone", "emergency_context": "Resumo" } `;
```

Elaborado pelo próprio autor (2026).

Código 13 – Assistant Prompt contextual

```
assistantprompt.js

1 function assistantprompt(context) {
    return ` # Informações Horário/Dia: ${context.emergency.time} Rua:
2 ${context.location.street} Bairro: ${context.location.neighborhood}
Cidade: ${context.location.city} Estado: ${context.location.state} País:
${context.location.country} CEP: ${context.location.zip} `;
3 }
```

Elaborado pelo próprio autor (2026).

Código 14 – Montagem conteúdo multimodal

```
multimodal.js

1 const content = [];
2 if (user) content.push({ type: "text", text: user });
3 if (image) {
4     const ext = image.split(".").pop().toLowerCase();
5     const fmt = ext === "jpg" ? "jpeg" : ext;
6     content.push({ type: "image_url", image_url: { url: `data:image/
${fmt};base64,` + fileToBase64(image) } });
7 }
8 if (audio) content.push({ type: "input_audio", input_audio: { data:
fileToBase64(audio), format: "ogg" } });
```

Elaborado pelo próprio autor (2026).

4.4.1 Formato de Saída (JSON)

O System Prompt exige resposta exclusivamente em JSON com `service_name`, `phone_number` e `emergency_context`. O servidor valida esses campos antes de encaminhar ao cliente, retornando erro 500 se inválido.

4.5 Integração com Modelos de Linguagem

Utilizou-se o OpenRouter como gateway unificado. Comunicação via `POST https://openrouter.ai/api/v1/chat/completions` com `model: online`, `stream: false` e autenticação `Bearer OPENROUTERAPIKEY` via `dotenv`. São registrados `response_time_ms`, `tokens` e `cost` para benchmark.

Código 15 – Chamada OpenRouter

```
openrouter.js
1  require("dotenv").config();
2  const dotenv = process.env;
3  const openrouterURL = "https://openrouter.ai/api/v1/chat/completions";
4  const messages = [
5    { role: "system", content: system },
6    { role: "assistant", content: assistant },
7    { role: "user", content }
8  ];
9  const body = { model: `${model}:online`, messages, stream: false };
10 const response = await fetch(openrouterURL, {
11   method: "POST",
12   headers: { Authorization: `Bearer ${dotenv.OPENROUTERAPIKEY}`,
13     "Content-Type": "application/json" },
14   body: JSON.stringify(body)
15 });
```

Elaborado pelo próprio autor (2026).

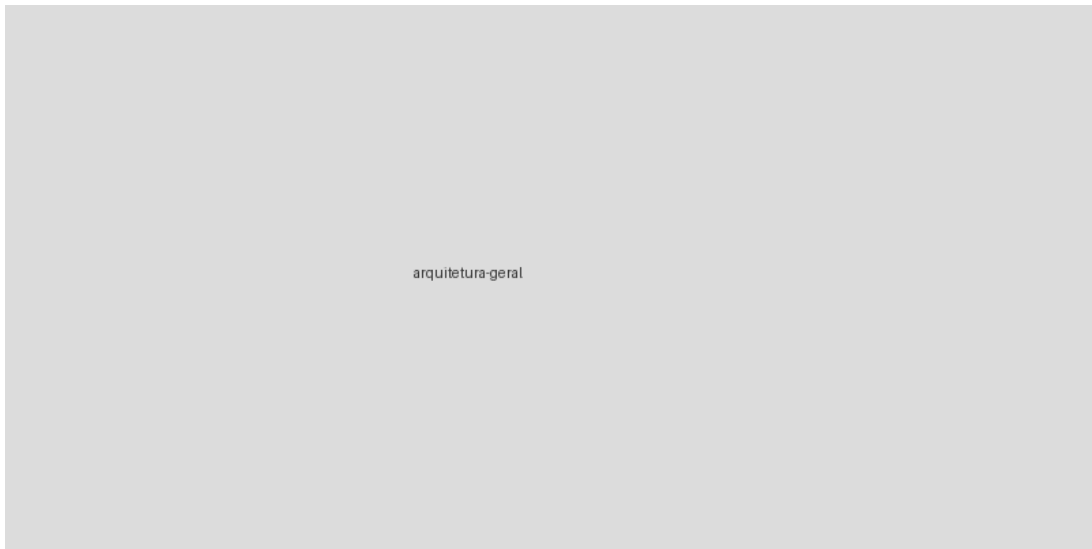
Código 16 – Tratamento resultado e validação JSON

```
result.js
1  const startedAt = Date.now();
2  const data = await response.json();
3  const responseTime = Date.now() - startedAt;
4  const result = {
5    content: extractJson(data.choices?.[0]?.message?.content),
6    cost: data.usage?.cost ?? 0,
7    response_time_ms: responseTime,
8    prompt_tokens: data.usage?.prompt_tokens ?? 0,
9    completion_tokens: data.usage?.completion_tokens ?? 0,
10   total_tokens: data.usage?.total_tokens ?? 0
11 };
12 const rec = result?.content;
   const valid = typeof rec?.service_name === "string" && typeof
13 rec?.phone_number === "string" && typeof rec?.emergency_context ===
   "string";
14 if (!valid) return res.status(500).json({ success: false, reason:
   "Invalid model response" });
   return res.status(200).json({ success: true, service_name:
15 rec.service_name, phone_number: rec.phone_number, emergency_context:
   rec.emergency_context });
```

Elaborado pelo próprio autor (2026).

A arquitetura completa (cliente → `/help` → geocodificação/prompt → Open-Router → validação) centraliza credenciais no backend e permite trocar modelo sem alterar o frontend, conforme Figura 8.

Figura 8 – Arquitetura geral do sistema



Elaborado pelo próprio autor (2026).

5 Análise dos Resultados

Com a prova de conceito desenvolvida, avaliou-se o comportamento dos sete modelos (GPT-5.5, Claude Opus 4.8, Llama 4 Maverick, Qwen 3.6 Plus, Gemini 3.5 Flash, Nemotron 3 Nano Omni 30B, North Mini Code) via OpenRouter em quatro cenários: (1) texto, (2) texto+imagem, (3) áudio, (4) áudio+imagem — todos com coordenadas/horário.

5.1 Modelos e Cenários

Os cenários representam: veículo no acostamento à noite (texto), árvore sobre fiação após tempestade (texto+imagem), gato atropelado (áudio) e residência alagada (áudio+imagem).

Tabela 2 – Cenários testados	
Cenário	Entrada
Primeiro	Texto
Segundo	Texto e Imagem
Terceiro	Áudio
Quarto	Áudio e Imagem

Elaborado pelo próprio autor (2026).

5.2 Coleta e Comparações

Registrados `cost`, `response_time_ms`, `prompt_tokens`, `completion_tokens` e erros. Exemplo Nemotron em Tabela 3.

Tabela 3 – Exemplo de métricas por modelo							
Parâmetro	GPT-5.5	Claude	Llama	Qwen	Gemini	Nemo- tron	North
cost	0.107	0.114	0.006	0.011	0.012	0.005	0.005
response_time_ms	44059	11028	2634	47734	10398	3910	6445
prompt_tokens	15370	19821	3658	3837	524	3982	3655
completion_tokens	972	227	48	2496	1336	351	497

Elaborado pelo próprio autor (2026).

Comparação isolada (uma requisição por modelo, sem transcrição auxiliar):

Tabela 4 – Comparação por critérios — modelo isolado

Critério	GPT	Claude	Llama	Qwen	Gemini	Nemo- tron	North
Texto	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso
Texto+Imagem	Sucesso	Falha	Sucesso	Sucesso	Sucesso	Sucesso	Falha
Áudio	Falha	Falha	Falha	Falha	Sucesso	Falha	Falha
Áudio+Imagem	Falha	Falha	Falha	Falha	Sucesso	Falha	Falha
JSON	Parcial	Parcial	Parcial	Parcial	Com- pleto	Parcial	Parcial
Resposta	Boa	Boa	Boa	Boa	Exce- lente	Regular	Boa

Elaborado pelo próprio autor (2026).

Apenas Gemini 3.5 Flash atendeu todos os cenários nativamente. Segunda análise (ecossistema sequencial: áudio→transcrição→classificação) viabilizou GPT e Llama para áudio via modelos auxiliares (GPT Audio, Muse Spark), aproximando cobertura de Gemini; Claude/Qwen permaneceram sem áudio, North só texto.

Tabela 5 – Viabilidade por ecossistema (composição sequencial)

Critério	GPT	Claude	Llama	Qwen	Gemini	Nemo- tron	North
Texto	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso
Texto+Imagem	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Sucesso	Falha
Áudio	Sucesso	Falha	Sucesso	Falha	Sucesso	Sucesso *	Falha
Áudio+Imagem	Sucesso	Falha	Sucesso	Falha	Sucesso	Sucesso *	Falha

Elaborado pelo próprio autor (2026).

Análises individuais detalhadas (recomendação, adequação, erros 404/400) confirmam diferenças de endpoints efetivamente disponíveis vs. capacidades anunciadas em interfaces de chat — ponto central do benchmark.

6 Conclusão

O Centro de Ajuda demonstrou viabilidade técnica como apoio ao direcionamento emergencial, integrando entrada multimodal, geocodificação, prompts e OpenRouter para entregar `service_name / phone_number / emergency_context` com redirecionamento `tel: .`

6.1 Avaliação dos Modelos

Apenas Gemini processou nativamente os quatro cenários; demais exigem composição ou ficam restritos a texto/imagem. A disponibilidade depende dos endpoints efetivamente expostos, não só do marketing da família.

6.2 Propostas Futuras

Aperfeiçoamento da aplicação (segurança, acessibilidade, offline com cache validado), ampliação de cenários (ruído, sotaques, baixa luz), avaliação de modelos locais (Llama/Gemma/Qwen) e validação com profissionais das áreas para curadoria de base de contatos por localização.

6.3 Considerações Finais

A prova de conceito integra interface, servidor, localização e LLMs, atendendo ao objetivo, mas não substitui centrais oficiais — é recurso intermediário para reduzir dúvidas e facilitar o acesso ao serviço adequado, com amplo campo de evolução.

REFERÊNCIAS

- ANTHROPIC, . *Claude Opus 4.8 Model Documentation*. Anthropic Docs, 2026. Disponível em: <<https://docs.anthropic.com/en/docs/about-claude/models>>. Citado na página 30.
- ASSOCIATION, E. E. N. *What is 112*. EENA, 2024. Disponível em: <<https://eena.org/about-112/whats-112-all-about/>>. Citado na página 18.
- ASSOCIATION, N. E. N. *911 Overview and Facts*. NENA, 2024. Disponível em: <<https://www.nena.org/page/911overviewfacts>>. Citado na página 18.
- BROWN, T.; MANN, B.; RYDER, N.; SUBBIAH, M.; KAPLAN, J.; DHARIWAL, P.; NEELAKANTAN, A.; SHYAM, P.; SASTRY, G.; ASKELL, A.; OTHERS, . *Language Models are Few-Shot Learners*. arXiv:2005.14165, 2020. Disponível em: <<https://arxiv.org/abs/2005.14165>>. Citado 2 vezes nas páginas 23 e 24.
- CLOUD, A. *Qwen API Platform*. Qwen, 2026. Disponível em: <<https://qwen.ai/apiplatform>>. Citado na página 30.
- CLOUD, A. *Qwen3-VL*. GitHub, 2026. Disponível em: <<https://github.com/QwenLM/Qwen3-VL>>. Citado na página 30.
- COHERE, . *North Mini Code 1.0*. Cohere Docs, 2026. Disponível em: <<https://docs.cohere.com/docs/north-mini-code-1.0>>. Citado na página 31.
- DOSOVITSKIY, A.; BEYER, L.; KOLESNIKOV, A.; WEISSENBORN, D.; ZHAI, X.; UNTERTHINER, T.; DEHGHANI, M.; MINDERER, M.; HEIGOLD, G.; GELLY, S.; OTHERS, . *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. arXiv:2010.11929, 2021. Disponível em: <<https://arxiv.org/abs/2010.11929>>. Citado na página 27.
- FOUNDATION, O. *Express - Fast, unopinionated, minimalist web framework*. Expressjs.com, 2024. Disponível em: <<https://expressjs.com/>>. Citado 2 vezes nas páginas 32 e 33.
- FOUNDATION, O. *Node.js Documentation*. Node.js, 2024. Disponível em: <<https://nodejs.org/>>. Citado na página 32.
- GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. Cambridge: MIT Press, 2016. Citado 4 vezes nas páginas 21, 22, 27 e 28.
- GOOGLE, . *Gemini 3.5 Flash Model Documentation*. Google AI for Developers, 2026. Disponível em: <<https://ai.google.dev/gemini-api/docs/models>>. Citado na página 30.
- GOOGLE, . *Google AI Platform*. Google AI, 2026. Disponível em: <<https://ai.google.dev/>>. Citado na página 30.
- IBM, . *What is Machine Learning*. IBM Topics, 2024. Disponível em: <<https://www.ibm.com/topics/machine-learning>>. Citado na página 20.

META, . *Llama 4 Maverick*. Meta AI, 2026. Disponível em: <<https://www.llama.com/models/llama-4/>>. Citado na página 30.

META, . *Llama API Overview*. Meta Developer Docs, 2026. Disponível em: <<https://llama.developer.meta.com/docs/overview/>>. Citado na página 30.

MITCHELL, T. *Machine Learning*. New York: McGraw-Hill, 1997. Citado na página 19.

MOSS, E. *History of Emergency Numbers*. Journal of Social History, 2017. Disponível em: <<https://doi.org/10.1093/jsh/shx101>>. Citado 2 vezes nas páginas 13 e 14.

MOZILLA, . *Fetch API*. MDN Web Docs, 2024. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API>. Citado 2 vezes nas páginas 32 e 33.

MOZILLA, . *MDN Web Docs*. Mozilla Developer Network, 2024. Disponível em: <<https://developer.mozilla.org/>>. Citado na página 31.

MOZILLA, . *Progressive Web Apps*. MDN Web Docs, 2024. Disponível em: <https://developer.mozilla.org/en-US/docs/Web/Progressive_web_apps>. Citado na página 31.

NVIDIA, . *Nemotron 3 Nano Omni 30B A3B Reasoning*. NVIDIA Build, 2026. Disponível em: <<https://build.nvidia.com/nvidia/nemotron-3-nano-omni-30b-a3b-reasoning>>. Citado na página 31.

OPENAI, . *GPT-4 Technical Report*. arXiv:2303.08774, 2023. Disponível em: <<https://arxiv.org/abs/2303.08774>>. Citado 3 vezes nas páginas 25, 26 e 28.

OPENAI, . *Prompt Engineering Guide*. OpenAI Platform Docs, 2024. Disponível em: <<https://platform.openai.com/docs/guides/prompt-engineering>>. Citado 2 vezes nas páginas 24 e 25.

OPENAI, . *GPT-5.5 Model Documentation*. OpenAI Developer Platform, 2026. Disponível em: <<https://developers.openai.com/api/docs/models/gpt-5.5>>. Citado 2 vezes nas páginas 29 e 30.

RADFORD, A.; KIM, J. W.; XU, T.; BROCKMAN, G.; MCLEAVEY, C.; SUTSKER, I. *Robust Speech Recognition via Large-Scale Weak Supervision*. arXiv:2212.04356, 2022. Disponível em: <<https://arxiv.org/abs/2212.04356>>. Citado na página 28.

RAPIDSOS, . *Unite Platform for Public Safety*. RapidSOS, 2025. Disponível em: <<https://wordpress-1533060-5913121.cloudwaysapps.com/public-safety/unite/>>. Citado na página 17.

RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach*. Berkeley: Pearson, 2020. Citado na página 19.

TELECOMUNICAÇÕES, A. N. D. *Numeração de Serviços de Emergência*. ANATEL, 2025. Disponível em: <<https://www.gov.br/anatel/pt-br/regulado/numeracao>>. Citado na página 14.

UNIJUI, R. C. \. S. -. *Conhecimento da População sobre Números de Emergência*. Revista Contexto \& Saúde, 2024. Disponível em: <<https://www.revistas.unijui.edu.br/index.php/contextoesaude/article/view/14835>>. Citado na página 14.

VASWANI, A.; SHAZEER, N.; PARMAR, N.; USZKOREIT, J.; JONES, L.; GOMEZ, A. N.; KAISER, L.; POLOSUKHIN, I. *Attention Is All You Need*. arXiv:1706.03762, 2017. Disponível em: <<https://arxiv.org/abs/1706.03762>>. Citado 3 vezes nas páginas 22, 23 e 28.

WEI, J.; WANG, X.; SCHUURMANS, D.; BOSMA, M.; ICHTER, B.; XIA, F.; CHI, E.; LE, Q.; ZHOU, D. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. arXiv:2201.11903, 2022. Disponível em: <<https://arxiv.org/abs/2201.11903>>. Citado na página 25.

Apêndice A – Códigos Complementares

Conteúdo adicional de implementação.

Anexo A – Portaria de Autorização

Conteúdo do anexo A.